

LAPORAN PROYEK AKHIR UAS KRIPTOGRAFI

CHAINPROOF

Sistem Verifikasi Integritas Dokumen Berbasis Blockchain Sederhana

Kelompok:

1. Ali Ma'sum Ajjawiyi (2488010010)
2. Fadheel Abdillah Husen (2488010013)
3. Muhammad Dhiyaul Haque (2488010041)

1. PENDAHULUAN

Latar Belakang Masalah

Dokumen merupakan sumber informasi dan aset penting yang dimiliki oleh setiap instansi, organisasi dan individu. Jika dokumen tidak dikelola dengan baik, maka data atau informasi yang dimiliki suatu instansi dapat hilang pada masa yang akan datang.

Perkembangan teknologi informasi saat ini dapat dengan mudah terjadinya pemalsuan dokumen yang dilakukan oleh pihak-pihak yang tidak berkepentingan. Bentuk pemalsuan barang biasanya dilakukan dengan memanipulasi isi dokumen dengan membuat tampilan desain dokumen yang sama dengan dokumen asli. Hal ini yang menjadikan kebutuhan dalam kerahasiaan informasi dan penjagaan validasi informasi semakin meningkat.

Perkembangan teknologi informasi memberikan pengolahan dokumen yang lebih efisien, yang berawal dari pengolahan dokumen secara manual menuju pengolahan dokumen berbasis digital yang mudah diakses. Pengolahan dokumen secara digital memainkan peran penting dalam meningkatkan kualitas layanan publik, transparansi, dan pengelolaan data jangka Panjang.

Tujuan Proyek:

1. Membangun sistem pendaftaran dan verifikasi dokumen berbasis blockchain sederhana
2. Mendemonstrasikan konsep chaining kriptografis, Proof of Work, dan deteksi manipulasi

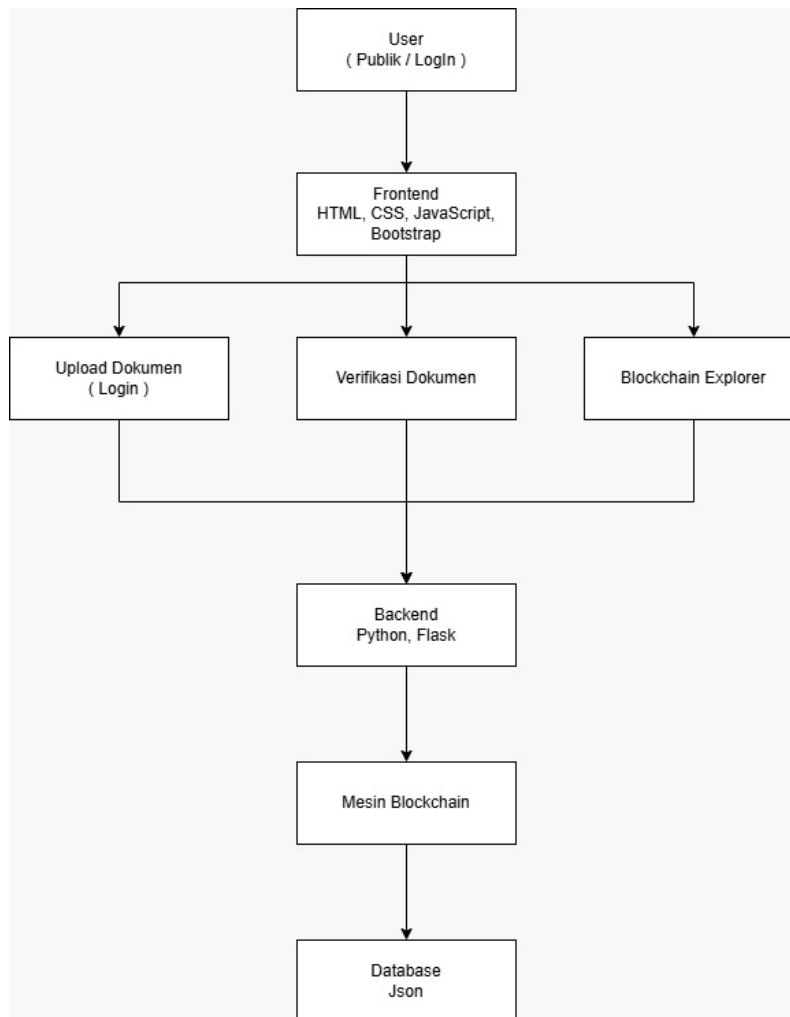
3. Menyediakan antarmuka web yang dapat digunakan oleh pengguna awam

Pengguna yang dituju:

Sistem ini dirancang untuk dua jenis pengguna. Pertama, pengguna terdaftar yaitu individu atau lembaga yang ingin mendaftarkan dokumen (ijazah, sertifikat, kontrak) ke chain sebagai bukti keberadaan pada waktu tertentu. Kedua, verifikator yaitu pihak mana pun yang ingin membuktikan keaslian dokumen tanpa memerlukan akses login, cukup dengan mengunggah file yang ingin diverifikasi.

2. ARSITEKTUR SISTEM

Arsitektur Sistem:



Komponen-komponen Sistem

1. User

User merupakan pihak yang berinteraksi dengan sistem RyuChain untuk melakukan registrasi maupun verifikasi dokumen digital. Sistem ini mengklasifikasikan pengguna ke dalam dua kategori, yaitu user publik dan user yang ter-autentikasi.

a. User Publik

Pengguna umum yang dapat mengakses fitur sistem tanpa memerlukan proses autentikasi (*login*). Fitur yang dapat diakses meliputi verifikasi dokumen, penelusuran rantai blok (*blockchain explorer*), serta pemeriksaan status *blockchain*.

b. User Ter-autentikasi

Pengguna yang telah melewati proses autentikasi (*login*) menggunakan akun yang terdaftar. Fitur yang dapat diakses mencakup seluruh fitur publik ditambah dengan kemampuan untuk mengunggah (*upload*) dokumen dan mendaftarkannya ke dalam *blockchain*. Pengguna dalam kategori ini memiliki hak istimewa untuk menambahkan blok baru ke dalam jaringan melalui proses unggah dokumen.

2. Frontend

Frontend merupakan antarmuka utama yang memfasilitasi interaksi antara pengguna dengan sistem. Lapisan presentasi ini dibangun menggunakan teknologi HTML, CSS, Javascript, Bootstrap, serta memanfaatkan template rendering dari Flask. Fitur yang tersedia pada frontend meliputi dashboard publik, formulir unggah dokumen, portal verifikasi dokumen, *blockchain explorer*, serta login dan registrasi. Secara fungsional, frontend bertugas untuk mengirimkan permintaan (*request*) kepada backend dan merender hasil pemrosesan tersebut untuk ditampilkan kepada pengguna.

3. Backend

Backend dikembangkan menggunakan framework Flask dengan bahasa pemrograman Python. Komponen ini berperan sebagai penghubung logika utama antara frontend dengan *blockchain engine*. Fungsi utama backend mencakup pengelolaan autentikasi pengguna, penanganan proses unggah berkas, kalkulasi nilai hash dokumen, eksekusi proses *blockchain*, validasi dokumen, serta pengelolaan *blockchain explorer*. Selain itu, backend mengimplementasikan manajemen sesi (*session login*) untuk membatasi akses unggah dokumen secara eksklusif hanya bagi pengguna yang telah terautentikasi.

4. Mesin Blockchain

Mesin blockchain merupakan lapisan inti sistem yang memproses seluruh operasi kriptografi dan mekanisme blockchain. Komponen ini terdiri atas:

- a. SHA-256, berfungsi untuk menghasilkan nilai hash yang unik dari dokumen fisik dan blok-blok di dalam rantai.
- b. RSA Digital Signature, digunakan untuk menandatangani hash dokumen menggunakan kunci privat dan memverifikasi keaslian data tersebut menggunakan kunci publik.
- c. Proof of Work (PoW), mekanisme konsensus sederhana yang mewajibkan sistem mencari nilai nonce tertentu hingga hash blok diawali dengan pola spesifik (misalnya "3910"). Fitur ini memastikan bahwa setiap blok membutuhkan usaha komputasi sebelum dapat ditambahkan ke rantai.
- d. Chain Validator, algoritma yang bertugas melakukan iterasi pada seluruh rantai blok untuk memverifikasi bahwa hash saat ini dan hash sebelumnya (previous hash) saling terhubung dengan benar, serta mendeteksi jika terjadi manipulasi data di dalam blok.

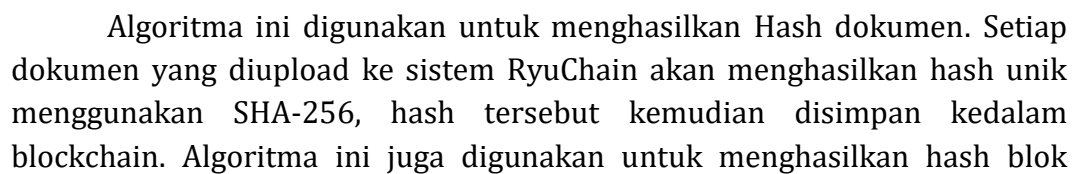
5. Database

Komponen ini bertindak sebagai media persistensi data (I/O).

Penyimpanan dibagi menjadi tiga bagian:

- a. Data Transaksional (`blockchain.json`), menyimpan seluruh ledger atau riwayat blok yang berisi hash dokumen, signature, dan metadata lainnya.
- b. Data Kredensial (`users.json`), menyimpan daftar pengguna yang telah terdaftar beserta password yang telah di-hash menggunakan `bcrypt`.
- c. Key Storage, direktori lokal untuk menyimpan Key Pair RSA (`private_key.pem` dan `public_key.pem`).
- d. Uploads, folder sementara untuk menyimpan dokumen yang diunggah pengguna sebelum diproses.

3. DESAIN KRIPTOGRAFI



blockchain dan juga digunakan untuk mendeteksi perubahan dari hash dokumen yang sudah disimpan ke dalam block.

Algoritma ini digunakan karena memiliki keamanan yang tinggi dimana pada setiap dokumen menghasilkan hash yang berbeda, selain itu SHA-256 sangat cocok untuk proses verifikasi integritas dokumen karena perubahan sekecil apapun pada dokumen akan menghasilkan hash yang berbeda, sehingga dapat mengatasi masalah pemalsuan dokumen.

Algoritma lain yang dapat digunakan dalam menghasilkan hash dokumen yaitu MD5, karena sangat cepat dan implementasinya yang sederhana. Namun tidak digunakan dalam sistem ini karena MD5 sangat rentan collision attack.

b. RSA Digital Signature

Algoritma RSA Digital Signature ini digunakan untuk proses penandatanganan dokumen digital, dimana hash dokumen akan ditandatangani oleh RSA Digital Signature menggunakan Private key untuk menghasilkan Signatur. Selain itu RSA Digital Signature digunakan memverifikasi keaslian data dokumen menggunakan Publik key.

RSA Digital Signature dipilih karena Algoritma yang populer dan cocok untuk Autentikasi dokumen dalam lingkup yang lebih kecil.

Algoritma lain yang dapat dipertimbangkan untuk digunakan yaitu ECDSA (Eliptic Curve Digital Signature Algoritm), algoritma ini lebih cepat dibandingkan dengan RSA Digital Signature, ukuran key yang lebih kecil, dan implementasi lebih kompleks.

c. Bcrypt

Pada sistem login dan register, proyek ini menggunakan BCrypt untuk hashing password pengguna. Password tidak disimpan dalam bentuk asli di database, melainkan diubah terlebih dahulu menjadi hash BCrypt.

Saat pengguna melakukan registrasi, password akan diproses menggunakan BCrypt yang secara otomatis menambahkan salt acak dan melakukan hashing berulang menggunakan cost factor tertentu. Hasil hash inilah yang kemudian disimpan ke file users.json. Saat login, password yang diinput pengguna akan dibandingkan dengan hash BCrypt menggunakan proses verifikasi BCrypt.

BCrypt dipilih karena dirancang khusus untuk keamanan password hashing. Berbeda dengan SHA-256 yang sangat cepat, BCrypt sengaja dibuat lebih lambat agar serangan brute force menjadi jauh lebih sulit dilakukan. Selain itu BCrypt memiliki salt otomatis sehingga lebih aman terhadap rainbow table attack.

Alternatif yang dipertimbangkan adalah SHA-256 untuk password hashing. Namun SHA-256 tidak dipilih karena terlalu cepat sehingga lebih rentan terhadap brute force attack. BCrypt dipilih karena jauh lebih aman untuk sistem autentikasi pengguna.

d. PoW (Proof of Work)

Algoritma Proof of Work (PoW) adalah mekanisme validasi pada blockchain yang digunakan untuk memastikan bahwa setiap block telah melalui proses verifikasi sebelum ditambahkan ke blockchain. Pada proyek ChainProof, PoW bekerja dengan cara menghitung hash block secara berulang menggunakan SHA-256 hingga ditemukan hash yang memenuhi target tertentu, seperti diawali beberapa angka nol. Proses ini dilakukan dengan mengubah nilai nonce secara terus-menerus sampai hash valid ditemukan.

PoW dipilih karena mampu menjaga integritas blockchain, mempersulit manipulasi data, serta mudah diimplementasikan untuk simulasi blockchain akademik.

Alternatif yang dipertimbangkan adalah Proof of Stake (PoS), namun tidak dipilih karena implementasinya lebih kompleks dibandingkan PoW.

4. IMPLEMENTASI SISTEM

Hash dokumen

```
def hash_document(file_path):  
    sha256 = (hashlib.sha256())  
    with open(file_path, "rb") as file:  
        while chunk := file.read(4096):  
            sha256.update(chunk)  
  
    return sha256.hexdigest()
```

SHA-256 digunakan untuk melakukan hashing dokumen atau data yang diupload.

RSA digital signature

```
with open( PRIVATE_KEY_PATH, "rb") as file:
```

```
private_key = (
    serialization
    .load_pem_private_key(file.read(), password=None)
)
```

Kode ini digunakan untuk memuat private key RSA yang digunakan dalam proses tanda tangan digital.

Sign dokumen

```
signature = (
    private_key.sign(document_hash.encode(),
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
            salt_length=(padding.PSS.MAX_LENGTH)
        ),
        hashes.SHA256()
    )
)

return (base64.b64encode(signature).decode())
```

Setelah mendapatkan private key, langkah selanjutnya adalah tanda tangan. Dokumen yang sudah di *hashing* ditanda tangani untuk membuktikan keaslian registrasi dokumen. Tanda tangan yang sudah didapatkan diubah ke bentuk Base64 agar bisa disimpan di file .JSON

PoW

```
target = "3910"

while not block.hash.startswith(target):

    block.nonce += 1

    block.hash = (block.calculate_block_hash())
```


validasi block dengan cara mencari nilai nonce tertentu agar hash block memenuhi target tertentu, yaitu prefix 3910. Sistem akan mencoba berbagai nilai nonce sampai menemukan hash valid. Tujuannya untuk meningkatkan integritas blockchain dan mempersulit manipulasi data karena perubahan pada block mengharuskan proses mining ulang.

Verifikasi tanda tangan

```
with open(PUBLIC_KEY_PATH, "rb") as file:

    public_key = (serialization.load_pem_public_key(file.read()))

    public_key.verify(

        base64.b64decode(signature),

        document_hash.encode(),

        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),

            salt_length=(padding.PSS.MAX_LENGTH)

        ),

        hashes.SHA256()

    )
```

Untuk memverifikasi tanda tangan, signature diubah dari Base64 ke bentuk biner lalu dicocokkan dengan publik key, jika cocok signature valid, jika tidak cocok maka InvalidSignature

Proses verifikasi data

```
for block in (blockchain.chain):

    if (block.document_hash == document_hash):

        signature_valid = (

            verify_signature(document_hash, block.owner_signature)

        )
```

Untuk memverifikasi data, langkah pertama yaitu *hashing* file yang ingin diverifikasi. Lalu mencari nilai hash dokumen yang sama yang ada di dalam blok, jika cocok langsung verifikasi tanda tangan.

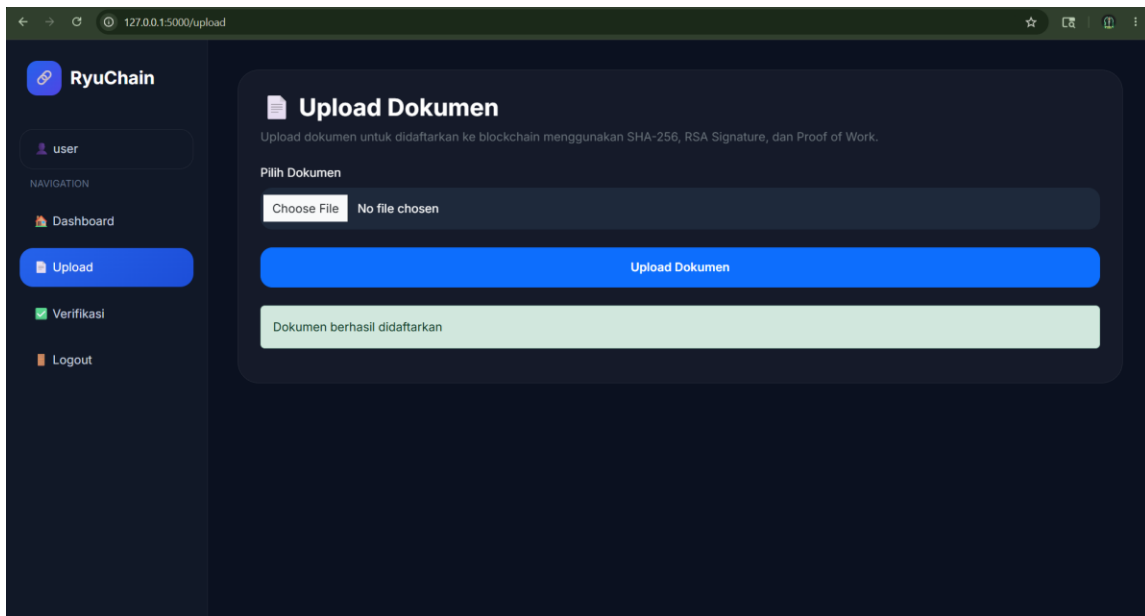
Password

```
def hash_password(password):  
    password_bytes = (password.encode())  
    salt = (bcrypt.gensalt())  
    hashed_password = (bcrypt.hashpw(password_bytes, salt))  
    return (hashed_password.decode())
```

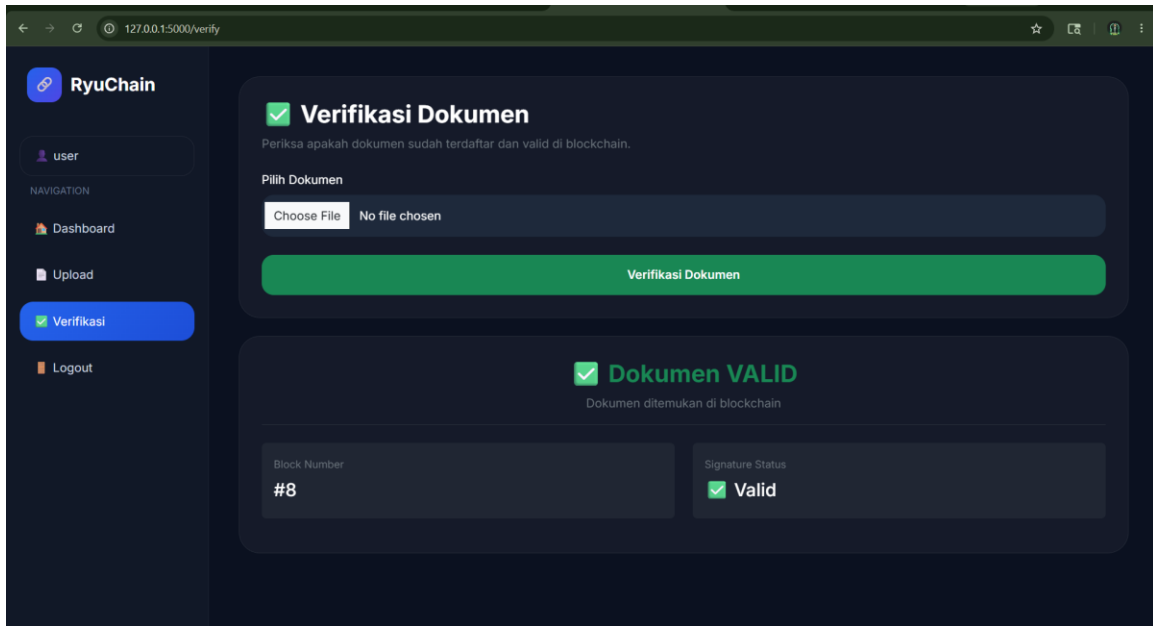
Password di-*hashing* menggunakan bcrypt, karena bcrypt dirancang untuk hashing password dan lebih aman dibandingkan dengan SHA-256.

5. PENGUJIAN SISTEM

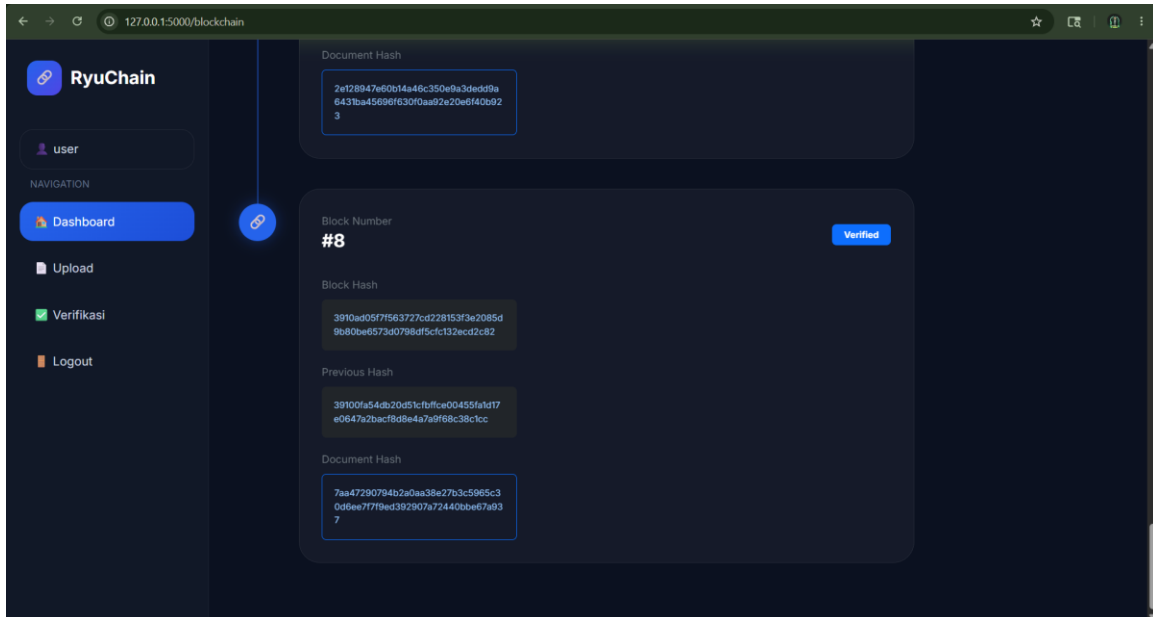
User mengunggah dokumen



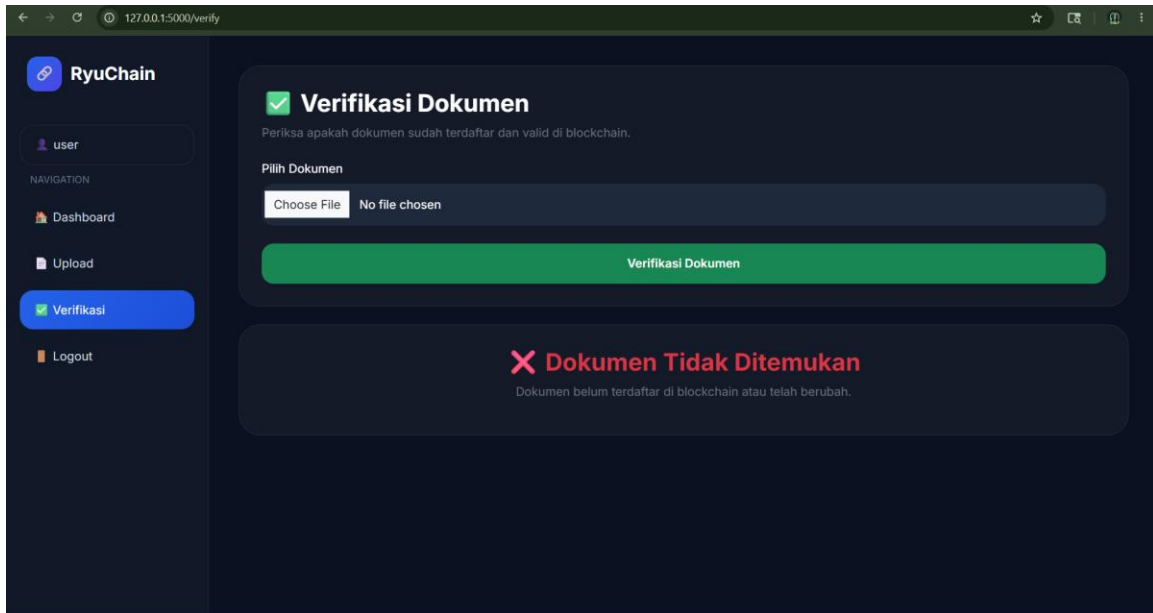
Verifikasi dokumen



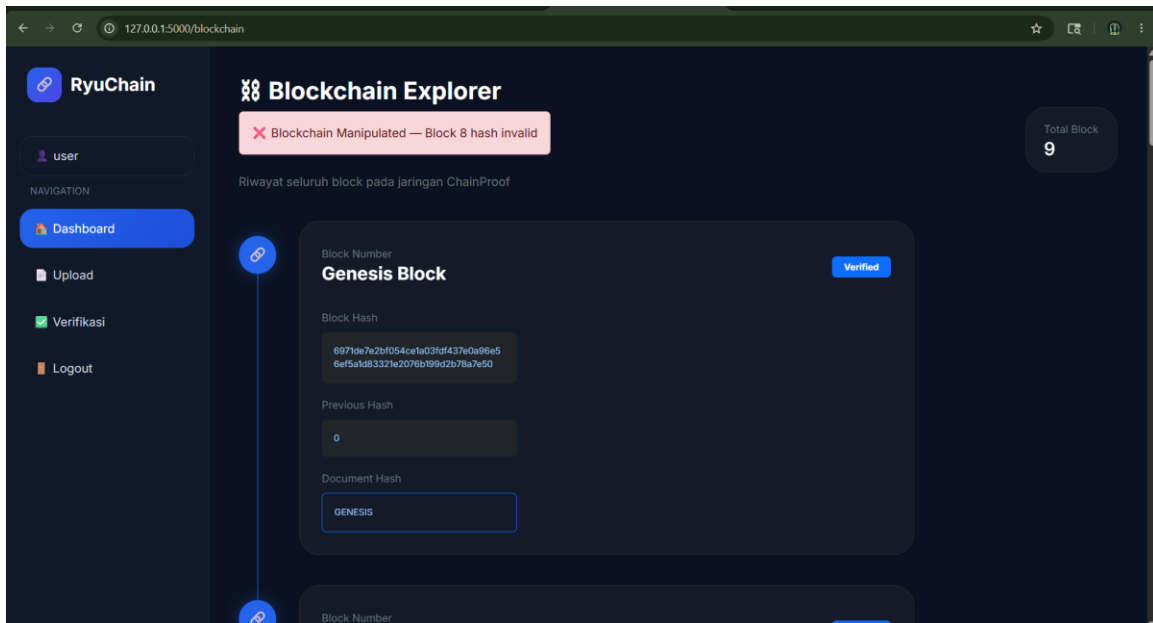
Tampil di block



Verifikasi dokumen yang belum di upload



Mengubah hash dokumen



6. KETERBATASAN DAN REFLEKSI

Dalam pengembangan sistem Chainproof ini, kami menyadari adanya batasan teknis yang perlu diperhatikan, di antaranya:

- a. Arsitektur penandatanganan terpusat (server side signing), dokumen milik pengguna yang berbeda ditandatangani oleh kunci yang sama milik server.
- b. Penyimpanan database menggunakan format JSON, metode ini memiliki keterbatasan dalam hal performa pembacaan data ketika jumlah blok di masa depan menjadi sangat besar (scalability issue) karena sistem harus memuat seluruh isi file ke dalam memori untuk melakukan iterasi pencarian.
- c. Penyimpanan dokumen asli masih lokal tersimpan di dalam folder uploads/, sehingga keamanan file sangat rentan.
- d. Belum menggunakan Merkle Tree.
- e. Belum Multi-chain yang terpisah.

Berdasarkan keterbatasan di atas, terdapat beberapa langkah pengembangan yang dapat dilakukan di masa depan untuk meningkatkan kualitas sistem:

- a. Implementasi User Based Digital Signature, Mengubah mekanisme penandatanganan agar setiap pengguna memiliki key pair RSA masing-masing. Sistem dapat mengelola penyimpanan key bagi setiap pengguna (misalnya disimpan di database dengan enkripsi tambahan), sehingga setiap dokumen memiliki "sidik jari" pemilik yang unik.
- b. Mengganti database dari penyimpanan JSON ke sistem database relasional seperti SQLite.
- c. Implementasi Merkle Tree.
- d. Implementasi Mullti-Chain.

Link demo: <https://youtu.be/DlpohKd1BIE>